DOMINI D'ESAME

Approfondimento D1/D5 · Sistemi multi-agente

Coordinatore e subagent, delega con Task, contesto isolato, strumenti per ruolo, provenienza, errori parziali e cicli di rifinitura.

11 · Materiale di studio Formato **Markdown**

> Appunti per la certificazione CCA-F (Domini 1 e 5, scenario *Multi-Agent Research System*). Spiegano come si progetta, si delega, si gestiscono errori e qualità in un sistema con un **coordinatore** e più **subagent**.
>
> ⚠️ Flag e nomi delle opzioni possono cambiare tra versioni. Per i dettagli aggiornati fa fede la documentazione ufficiale: **docs.claude.com** → **Agent SDK** → **Subagents** e **Claude Code** → **Subagents**.

Indice

1. Lo scenario d'esame
2. Architettura hub-and-spoke
3. Come si lanciano i subagent
4. Il contesto è isolato
5. Scrivere un buon prompt di delega
6. Parallelo o sequenziale
7. Privilegio minimo: gli strumenti per ruolo
8. Cosa restituisce un subagent
9. Gestione degli errori
10. Copertura, qualità e cicli di rifinitura
11. Fonti in conflitto e fedeltà delle citazioni
12. Sessioni nei sistemi multi-agente
13. Quando NON usare più agenti
14. Trappole e domande da esame

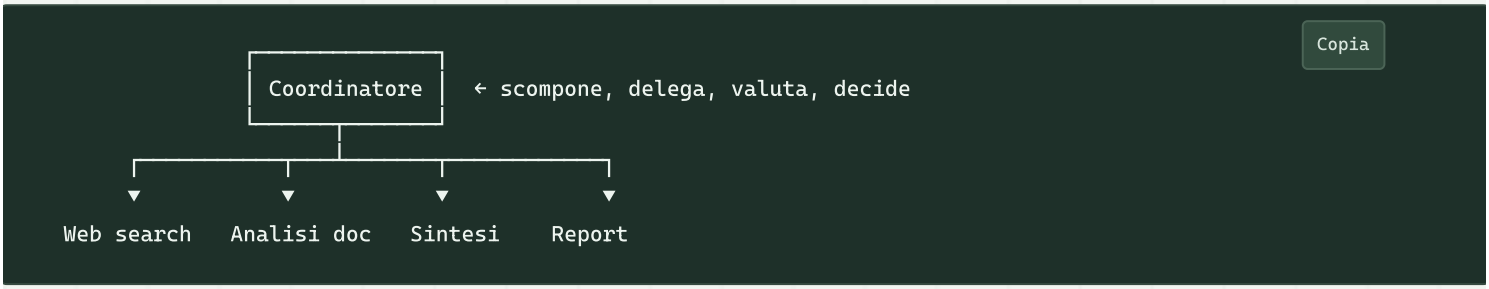
1. Lo scenario d'esame

> *Un agente coordinatore delega a subagent specializzati: uno cerca sul web, uno analizza documenti, uno sintetizza i risultati, uno genera il report. Il sistema produce report completi e con citazioni.*

Le domande di questo scenario girano quasi sempre attorno a cinque temi:

Tema	Domanda tipica
Delega	Perché il subagent non sa cosa fare / non viene invocato?
Latenza	Perché la ricerca è 4 volte più lenta del previsto?
Copertura	Perché il report è incompleto se ogni subagent ha lavorato bene?
Provenienza	Perché il report contiene affermazioni senza fonte?
Errori	Cosa fa un subagent quando una parte del lavoro fallisce?

2. Architettura hub-and-spoke



Regole:

- Il coordinatore **scompone** il compito in sottoargomenti, **delega**, **raccoglie** i risultati e **decide** i passi successivi.
- **Tutta la comunicazione passa dal coordinatore.** I subagent non si parlano direttamente.
- Perché? Il coordinatore deve **vedere** risultati ed errori per poter reagire (ritentare, colmare un buco, segnalarlo).

Trappola: "per ridurre la latenza, il subagent di ricerca manda i risultati direttamente alla sintesi". Sembra efficiente, ma se un passaggio fallisce **nessuno se ne accorge** e il report esce incompleto in silenzio.

3. Come si lanciano i subagent

Nell'**Agent SDK** (e in Claude Code):

- I subagent si definiscono nell'opzione `agents` (nome, descrizione, prompt, strumenti, eventualmente modello).
- Il coordinatore li invoca tramite lo strumento `Task`.
- `Task` **deve essere in** `allowedTools` **del coordinatore.** Se manca, il coordinatore non ha modo di delegare e **fa tutto da solo** con gli strumenti che ha.

```python
options = ClaudeAgentOptions(
    allowed_tools=["Task", "Read"],          # senza "Task" niente delega
    agents={
        "web-search": AgentDefinition(
            description="Cerca fonti sul web per un sottoargomento",
            prompt="Sei uno specialista di ricerca …",
            tools=["WebSearch", "WebFetch"],
        ),
        "synthesis": AgentDefinition(
            description="Sintetizza findings strutturati con citazioni",
            prompt="Sei uno specialista di sintesi …",
            tools=["Read"],                  # niente strumenti web
        ),
    },
)
```

Copia PYTHON

> 💡 La **descrizione** del subagent conta come la descrizione di un tool: il coordinatore la usa per decidere **a chi** delegare. Descrizioni vaghe → deleghe sbagliate.

---

4. Il contesto è isolato

È **la regola più testata** di tutto lo scenario.

- Un subagent parte con un **contesto nuovo**. Non vede la cronologia del coordinatore, i messaggi precedenti, i documenti "di cui abbiamo parlato".
- Sa **solo** quello che c'è nel prompt di delega (più il suo system prompt).
- Non esiste alcuna "memoria condivisa degli ultimi N turni", né un file di sessione che il subagent possa leggere.

Esempio d'esame:

> Il coordinatore delega con *"Analizza i documenti di cui abbiamo parlato ed estrai i risultati principali"*. Il subagent restituisce risultati sbagliati.
>
> ✅ **Causa:** il subagent non ha accesso al contesto del coordinatore. **Soluzione:** passare esplicitamente domanda di ricerca, documenti e formato di output.

> ❌ "Usa un modello più grande" — non risolve l'informazione mancante.

Vantaggio dell'isolamento: il lavoro verboso (decine di pagine lette) resta nel contesto del subagent; al coordinatore torna solo il risultato. È lo stesso principio di `context: fork` nelle skill e dell'*Explore* subagent in Claude Code.

---

5. Scrivere un buon prompt di delega

Un prompt di delega completo contiene:

Elemento	Esempio
Obiettivo	"Raccogli evidenze sull'effetto del lavoro da remoto sui prezzi delle case nelle città medie"
Domande da rispondere	"Variazione dei prezzi 2019–2024? Differenze centro/periferia?"
Input	Elenco dei documenti, risultati già trovati, vincoli emersi
Criteri di qualità	"Fonti primarie o istituzionali, dati dal 2019, almeno 3 fonti indipendenti"
Limiti	Budget di ricerche, aree da escludere
Formato di output	"Lista di findings: affermazione, URL, titolo, data, estratto"

Obiettivi, non procedure. Un prompt rigido ("Passo 1: cerca 'X market size'. Passo 2: apri i primi 3 risultati...") fallisce appena l'argomento non si adatta alla procedura. Descrivi **cosa** serve e **come riconoscere un buon risultato**, lascia al subagent la strategia.

Requisiti che cambiano a metà. Se dopo il primo giro emergono nuovi vincoli (es. "solo fonti dopo il 2020"), il coordinatore li deve **includere esplicitamente** nelle deleghe successive: i subagent non li "sentono".

---

6. Parallelo o sequenziale

- **Sottoargomenti indipendenti** → il coordinatore emette **più chiamate `Task` nella stessa risposta**: i subagent girano in parallelo e la latenza scende a quella del più lento.
- **Dipendenze reali** (la sintesi ha bisogno dei risultati della ricerca) → sequenziale.

Sintomo	Causa	Soluzione
Latenza = 4× una singola ricerca	Una `Task` alla volta, aspettando ogni risultato	Tutte le `Task` indipendenti nella stessa risposta

❌ Trappole: "unisci i 4 sottoargomenti in un'unica chiamata lunga" (resta sequenziale e mescola i temi); "riduci `max_tokens`" (tronca gli output, non toglie la serializzazione).

7. Privilegio minimo: gli strumenti per ruolo

Ogni subagent riceve **solo gli strumenti del suo ruolo**:

Subagent	Strumenti tipici	Da NON dare
Web search	WebSearch, WebFetch	Write, Bash
Analisi documenti	Read (+ eventuale OCR/parsing)	WebSearch
Sintesi	nessuno o Read	WebSearch (aggiungerebbe fonti non verificate)
Report	Write (per il file finale)	Bash, WebSearch

Perché: meno strumenti = scelta più precisa, meno comportamenti fuori ruolo, più sicurezza. Un'istruzione nel prompt ("usa solo gli strumenti del tuo ruolo") è probabilistica; togliere lo strumento è deterministico.

Eccezione ragionata: se la sintesi deve spesso verificare un singolo fatto (una data, una cifra) e il giro via coordinatore costa troppa latenza, le si può dare uno **strumento di verifica ristretto** per i casi semplici. Le ricerche complesse continuano a passare dal coordinatore.

❌ Trappola: "dai alla sintesi tutto il toolset di ricerca ed elimina il subagent di ricerca" — distrugge la specializzazione.

8. Cosa restituisce un subagent

Risultati condensati e strutturati

Non il testo integrale delle pagine (25 pagine × 6.000 parole saturano il contesto del coordinatore e peggiorano le sue decisioni), ma **findings strutturati**.

Provenienza

Ogni affermazione deve viaggiare con la sua fonte:

JSON Copia

```
{
  "claim": "I prezzi delle case nei sobborghi sono aumentati del 14% tra 2020 e 2022",
  "source_url": "https:// ... ",
```

```
  "source_title": "...",
  "publication_date": "2023-05-10",
  "data_period": "2020-2022",
  "excerpt": "Suburban home prices rose 14% between 2020 and 2022 ..."
}
```

- Senza l'associazione **affermazione ↔ fonte**, la sintesi non può citare con precisione.
- Una sezione "Fonti" con tutti gli URL visitati **non basta**: non dice quale fonte sostiene quale affermazione.

Metadati temporali

La **data della fonte** e il **periodo dei dati** sono parte della provenienza. Senza date, valori del 2016, 2019 e 2024 sembrano contraddittori invece che un trend.

---

9. Gestione degli errori

Fallimento parziale

Il subagent di analisi riceve un PDF di 300 pagine; l'OCR fallisce sulle pagine 140–180.

Comportamento	Giudizio
Restituisce l'analisi come se fosse completa	❌ nasconde il buco
Scarta tutto e restituisce un errore generico	❌ butta via lavoro valido
Inventa il contenuto mancante dal contesto	❌ fabbricazione
Risultati parziali + metadati: quali pagine, perché, cosa potrebbe mancare	✅ il coordinatore può ritentare o segnalare

Risultato vuoto vs errore di accesso

"Nessuna fonte trovata" può significare due cose molto diverse:

Caso	Cosa restituire	Cosa fa il coordinatore
Ricerca riuscita, 0 risultati	Risultato **valido** vuoto + query provate	Riformula o conclude che il tema è poco coperto
API irraggiungibile, timeout	**Errore** con tipo e se è ritentabile	Ritenta o segnala il buco

❌ Trattarli allo stesso modo porta a scrivere nel report "non esiste letteratura" quando in realtà la ricerca non è mai avvenuta.

Chi decide

Il subagent **riporta**, il coordinatore **decide**: ritentare, delegare di nuovo, oppure dichiarare il buco nel report finale.

---

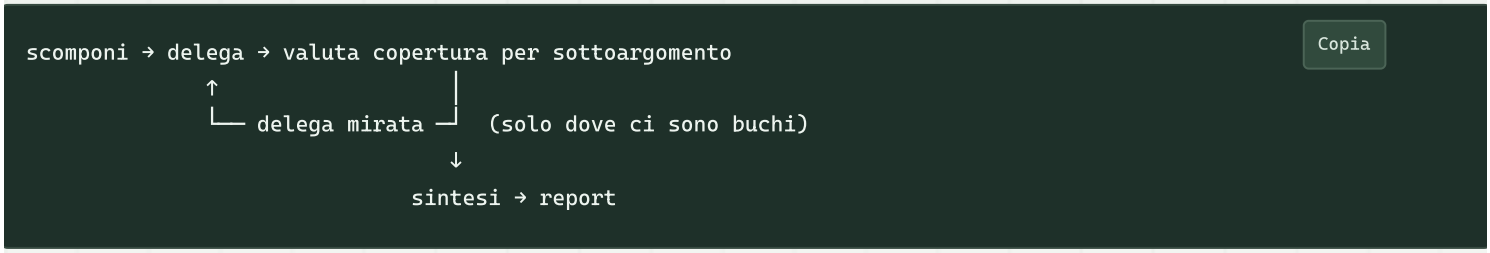
10. Copertura, qualità e cicli di rifinitura

Report incompleto con subagent "perfetti"

Se ogni subagent ha fatto esattamente quello che gli era stato chiesto e il report è comunque incompleto, la causa è **a monte**: la **scomposizione del coordinatore** era troppo stretta.

❌ Spostare la colpa su chi ha lavorato correttamente (es. "la ricerca doveva allargare le query da sola").

Ciclo di rifinitura



- Il coordinatore valuta **copertura** e **qualità delle fonti** per ogni sottoargomento.
- Dove ci sono buchi, delega **ricerche mirate**; dove la copertura è buona, non spreca risorse.

Criteri di arresto

Un ciclo senza limiti può andare avanti per 30 round e sforare il budget. Servono criteri espliciti:

- obiettivi di copertura per i sottoargomenti richiesti;
- numero massimo di round;
- budget (token, costo, tempo);
- alla fine, **segnalare i buchi rimasti** invece di nasconderli.

❌ Trappole: "tre round fissi per ogni argomento" (spreco dove non serve); "continua finché il contesto è pieno"; "elimina il ciclo e fai un solo round".

---

11. Fonti in conflitto e fedeltà delle citazioni

Fonti in conflitto

Fonte A (governo, 2023): tasso di sfitto 12%. Fonte B (settore, 2024): 19%.

Opzione	Giudizio
Riportare solo il più recente	❌ nasconde il conflitto
Fare la media (15,5%)	❌ numero che nessuna fonte riporta
Preferire sempre la fonte governativa	❌ regola arbitraria
Escludere la metrica	❌ perde informazione
Riportare entrambi con fonte, data e metodo, annotando il conflitto	✅

Spesso la differenza si spiega con **date** o **metodologie** diverse.

Fedeltà delle citazioni

Il report dice "il lavoro da remoto ha ridotto gli affitti commerciali del 23%"; la fonte dice "in **alcune** aree centrali gli affitti sono calati **fino al** 23%".

✅ **Passo di verifica:** confrontare ogni affermazione della bozza con l'**estratto** salvato della sua fonte e segnalare quelle con portata o qualificatori diversi. È possibile solo se la provenienza è stata conservata (sezione 8).

❌ Disclaimer generici ("le cifre possono essere approssimate"), eliminare tutte le percentuali, accettare solo fonti con cifre "esatte".

Niente contenuto senza fonte

La sintesi **non colma i buchi con la propria conoscenza**: produrrebbe affermazioni non citate, violando il requisito di report con citazioni.

12. Sessioni nei sistemi multi-agente

Situazione	Strumento
Il processo si è interrotto, si vuole continuare	`--resume` / `resume` con l'ID di sessione
Si è ripreso ma alcune fonti sono cambiate	`resume` + **dire esplicitamente** all'agente cosa è cambiato, così rifà solo quelle parti
Esplorare due strategie alternative dalla stessa base	`fork_session`: rami indipendenti con il contesto comune

Situazione	Strumento
Contesto degradato dopo lunghe esplorazioni	Salvare le scoperte in un **file scratchpad** e ripartire con una sessione nuova

❌ Trappole: eseguire le due strategie una dopo l'altra nella stessa sessione (la seconda è influenzata dalla prima); ripartire da zero buttando ore di lavoro valido; riprendere "così com'è" con analisi ormai obsolete.

Approfondimento: Sessioni, resume, fork e subagent.

13. Quando NON usare più agenti

Più agenti significa più costo, più latenza e più punti di errore. Il coordinatore deve **scegliere dinamicamente** quali subagent servono:

- *"Quando è stato firmato il CHIPS Act?"* → una sola ricerca, nessuna pipeline completa.
- *"Analisi completa dei rischi nella supply chain dei semiconduttori"* → pipeline completa.

❌ Trappole: far scegliere all'utente quale applicazione usare; cache dei report completi (aiuta solo le domande identiche); ridurre `max_tokens` a tutti.

14. Trappole e domande da esame

Opzione che sembra giusta	Perché è sbagliata
"Il subagent di ricerca doveva ampliare le query da solo"	La copertura dipende dalla scomposizione del coordinatore
"La sintesi colma i buchi con la propria conoscenza"	Contenuto senza citazioni
"Fai la media dei valori in conflitto"	Inventa un numero
"I subagent condividono la memoria degli ultimi N turni"	Meccanismo inesistente: il contesto è isolato
"Aggiungi `Read` al subagent così legge la sessione del coordinatore"	Meccanismo inesistente
"Delega solo quando il contesto supera il 50%"	Meccanismo inesistente; il problema è `Task` mancante
"Dai a tutti i subagent tutti gli strumenti, per flessibilità"	Viola il privilegio minimo
"Scrivi nel prompt: usa solo gli strumenti del tuo ruolo"	Probabilistico: togli gli strumenti
"Subagent che comunicano direttamente per ridurre la latenza"	Nessuno vede gli errori

Opzione che sembra giusta	Perché è sbagliata
"Unisci tutti i sottoargomenti in un'unica chiamata"	Resta sequenziale, mescola i temi
"Restituisci le pagine complete, così nulla va perso"	Satura il contesto del coordinatore
"Aggiungi una sezione Fonti con tutti gli URL visitati"	Non collega affermazioni e fonti
"Il report generator rifà la ricerca per trovare una fonte a ogni affermazione"	Fonti trovate a posteriori, che magari non sostengono l'affermazione
"Continua in silenzio: l'87% delle pagine basta"	Nasconde un buco
"Usa un modello più grande per il subagent"	Non risolve informazioni mancanti nel prompt di delega

Parole chiave nella domanda → risposta probabile

Se leggi...	Pensa a...
"ogni subagent ha lavorato correttamente, ma..."	scomposizione del coordinatore
"i subagent non vengono mai invocati"	`Task` mancante in `allowedTools`
"i documenti di cui abbiamo parlato"	contesto isolato → passa tutto esplicitamente
"la latenza è N volte..."	`Task` multiple nella stessa risposta
"il contesto del coordinatore si riempie"	findings condensati e strutturati
"affermazioni senza fonte"	provenienza: claim + URL + data + estratto
"fonti in disaccordo"	riporta entrambe, annota il conflitto
"il ciclo continua per 30 round"	criteri di arresto espliciti
"fallisce su una parte del documento"	risultati parziali + metadati sul buco
"due strategie alternative dalla stessa base"	`fork_session`